AIN4101: Machine Learning II

Lecture 2: Text Representation and Word Embeddings

Dr.Badrya Dahy

Badr University in Assiut (BUA)
School of Artificial Intelligence & Data Management

Week 2, Spring 2026

Outline

1. Why Representation Is Hard
2. Classical Sparse Representations
3. Count-Based Distributional Representations
4. Prediction-Based Word Embeddings
5. Beyond Words: Documents and Context
6. Summary

Learning Objectives - Today

By the end of this lecture, you will be able to:

1. Explain why text is a hard input and why representation matters.
2. Compare one-hot, Bag of Words, n-grams, and TF-IDF.
3. Explain co-occurrence and PPMI as count-based distributional methods, and LSA/SVD as a dense low-dimensional compression.
4. Differentiate count-based from prediction-based embeddings.
5. Describe Word2Vec (CBOW, skip-gram, negative sampling), GloVe, and fastText.
6. Build document vectors from word vectors and explain why contextual models followed.
7. Choose a representation for a given task.

Key Takeaway

Lecture 1 covered preprocessing and tokenization. Today we study how tokens become vectors, and why each method replaced the previous one.

Where Lecture 2 Starts

raw text $\rightarrow$ tokens/IDs $\rightarrow$ representation $\rightarrow$ model $\rightarrow$ prediction

- After tokenization we hold tokens or IDs, but models need meaningful numerical features.
- A representation decides which information the model can see and which is lost.

Important Note

Representation is part of model design: it defines the input geometry the learner sees.

Why Text Is a Hard Input

ML Requirement

- Fixed-size numerical arrays.
- Text is symbolic, variable-length, and compositional.

Language Difficulty

- Huge vocabularies.
- Synonymy: *good*, *great*.
- Polysemy: *bank*.
- Order: “dog bites man” vs. “man bites dog”.

Vocabulary Size and Zipf's Law

- Word frequency is heavy-tailed: $f(r) \propto 1/r$ for rank r.
- A few words (*the, of, and*) dominate; most words are rare.
- Consequences: very large vocabularies, very sparse features, many unseen words (OOV).

Key Takeaway

Sparsity is structural, not accidental. Any representation must cope with rare and unseen words.

Historical Timeline: How Text Representation Evolved

Period	Milestone	What problem it addressed
1954	Harris: distributional structure	Meaning can be inferred from patterns of linguistic context.
1957	Firth: distributional hypothesis	Similar contexts suggest related meanings.
1960s	Salton: vector-space IR / BoW	Represent documents numerically for retrieval and comparison.
1972	Spärck Jones: IDF / TF-IDF	Reduce the influence of terms that occur in many documents.
1990	LSA / SVD	Discover lower-dimensional latent structure beyond exact word matching.
2003	Bengio neural language model	Learn distributed word representations jointly with prediction.
2013	Word2Vec	Make predictive word-representation learning efficient at very large scale.
2014	GloVe	Learn dense vectors from explicit corpus-wide co-occurrence statistics.
2017	fastText	Add subword information to improve morphology and unseen-word handling.
2018+	ELMo, BERT	Produce context-dependent representations instead of one vector per word type.

Important Note

There is no single inventor of "text representation." The field evolved through information retrieval, distributional linguistics, statistical methods, and neural representation learning.

How to Judge a Representation

- **Dimensionality:** compact enough for efficient learning?
- **Sparsity:** mostly zeros or dense?
- **Similarity:** are similar words/documents close?
- **Order and context:** is word order or sense captured?
- **OOV handling:** what happens with unseen words?
- **Cost and interpretability.**

Important Note

We will compare every method against these criteria in the final comparison table.

One-Hot Encoding

Idea

Vector length equals vocabulary size $|V|$; exactly one entry is 1.

- **Pros:** exact identity, simple; basis of token IDs and embedding lookup.
- **Cons:** huge and sparse; no similarity.
- For any two different words, $\mathbf{u} \cdot \mathbf{v} = 0$ and $\|\mathbf{u} - \mathbf{v}\| = \sqrt{2}$.

Example

cat and *dog* are exactly as far apart as *cat* and *car*.

Next: we need to represent whole documents.

Bag of Words (BoW)

Idea

A document is the vector of term counts over the vocabulary; order is discarded.

	deep	fun	is	learning	machine	powerful
D1: machine learning is fun	0	1	1	1	1	0
D2: machine learning is powerful	0	0	1	1	1	1
D3: deep learning is powerful	1	0	1	1	0	1

- **Pros:** simple, fast, strong baseline for spam, sentiment, topics.
- **Cons:** loses order, frequent words dominate, high-dimensional.
- In practice, document-term matrices are stored sparsely; fit the vocabulary on the training set only to avoid data leakage.

N-grams: Recovering Local Order

- An n-gram is a contiguous sequence of n tokens.
- Bigrams capture phrases such as “not good” and “machine learning”.
- Number of features grows roughly like $|V|^n$ in the worst case.

Example

“not good at all”: bigrams = not good, good at, at all.

Important Note

N-grams are a partial fix: more context, much larger and sparser space.

TF-IDF: From Frequency to Informativeness

- Raw counts cannot separate informative words from merely common ones.
- TF rewards frequency in the document; IDF penalizes frequency across the corpus.

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t), \qquad \mathrm{idf}(t) = \ln \frac{1+n}{1+\mathrm{df}(t)} + 1$$

- Vectors are usually L2-normalized so document length does not dominate.
- Classic form: $\mathrm{idf}(t) = \log(n/\mathrm{df}(t))$; libraries smooth it to avoid division by zero.

TF-IDF Worked Example

Corpus D1–D3 from the BoW slide, $n = 3$. Smoothed IDF with natural log:

Term	df	idf	tf-idf in D1
learning, is	3	1.000	1.000
machine	2	1.288	1.288
fun	1	1.693	1.693

L2 norm of D1: $\sqrt{1.693^2 + 1.288^2 + 1 + 1} \approx 2.554$.
Normalized D1: fun 0.663, machine 0.504, is 0.392, learning 0.392.

Key Takeaway

The rarest term (*fun*) gets the largest weight; ubiquitous words get the smallest.

- After L2 normalization, cosine similarity is a natural way to compare TF-IDF document vectors; TF-IDF with a linear model remains a strong, interpretable baseline.

The Shared Weakness of Sparse Representations

Strengths

- Simple, transparent, fast.
- Strong baselines; scale via sparse algebra.

Weaknesses

- Dimension grows with $|V|$.
- *good* and *excellent* are unrelated columns.
- Context-free; order ignored or partial.

Important Note

What the model learns about *good* cannot transfer to *great*. We need a space where similarity is visible.

Distributional Hypothesis

Firth, 1957

"You shall know a word by the company it keeps."

- Words in similar contexts have similar meanings.
- *doctor*, *nurse*, *hospital* share contexts; *bank*, *loan*, *account* share others.
- This single idea motivates both count-based distributional representations and prediction-based embeddings.

Two Families of Distributional Representation Learning

Frequency / Count-Based

- Start from observed frequency or co-occurrence statistics.
- Co-occurrence matrices, PPMI, LSA/SVD, and GloVe.
- Core question: "Which words occur together, and how often?"

Prediction-Based

- Learn vectors by solving a neural prediction task.
- Neural language models, CBOW, and skip-gram.
- Core question: "Can this representation predict a word or its context?"

Important Note

BoW and TF-IDF are classical sparse count-based document representations. They are useful precursors, but they are not dense word embeddings in the Word2Vec/GloVe sense.

Co-occurrence Matrix

- Cell (i, j) counts how often word j appears within a window around word i.
- Rows are word vectors; similar rows mean similar contexts.

	hot	tasty	river
coffee	10	8	0
tea	9	7	0
bank	0	1	6

Important Note

Coffee and tea have similar rows. But the matrix is still $|V| \times |V|$, sparse, and dominated by frequent words.

- A common refinement is **PPMI**, which asks whether a word-context pair occurs more often than chance: $\mathrm{PPMI}(w, c) = \max\left(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0\right)$.
- PPMI reweights raw counts before dimensionality reduction such as SVD/LSA.

LSA / SVD: Latent Semantic Analysis

$$X \approx U_k \Sigma_k V_k^\top$$

- Factorize a term-document or word-context matrix; keep the top k singular values.
- Rows of $U_k \Sigma_k$ give dense k-dimensional word vectors (typically 100–300).
- Related words move closer even without sharing documents.
- Choose k by explained variance or downstream validation.

Pros: dense, captures latent topics, closed-form.

Cons: costly on large corpora, hard to update, not contextual.

Neural Language Model (Bengio et al., 2003)

previous words $\rightarrow$ embedding layer $\rightarrow$ neural network $\rightarrow$ next word

- Word vectors are learned jointly with a model that predicts the next word.
- Replaces hand-designed features with representations learned from raw text.
- Limitation: expensive softmax over the whole vocabulary made training slow.

Key Takeaway

Representation learning: train on a language task, then reuse the learned internal vectors.

Dense Embeddings and Cosine Similarity

- A dense embedding maps a word to a vector of typically 50–300 real numbers.
- Cosine similarity compares direction, not length.
- Nearest neighbours reveal related words, and also dataset bias.

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} \in [-1, 1]$$

Example

Expected: $\cos(\textit{cat}, \textit{kitten})$ high, $\cos(\textit{cat}, \textit{algorithm})$ low.

The Embedding Layer Is a Lookup Table

- Matrix $E \in \mathbb{R}^{|V| \times d}$: one row per token ID.
- With one-hot $\mathbf{x}$: $\mathbf{x}^\top E$ selects one row, so lookup equals one-hot times matrix.
- Implementations index directly instead of multiplying.
- The gradient is non-zero only for rows used in the batch.

Key Takeaway

One-hot identifies the token; the embedding matrix turns identity into a trainable dense vector.

Word2Vec Overview (Mikolov et al., 2013)

- A shallow network trained on a self-supervised task built from raw text with a context window.
- The prediction task is not the goal; the learned matrix is.
- Two matrices: input vectors v_w (used as embeddings) and output vectors u_w.

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

- The denominator sums over $|V|$ words, which is the computational bottleneck.
- Key choices include context-window size, embedding dimension, negative samples, minimum word count, and training epochs; their best values depend on the corpus and downstream task.

CBOW and Skip-gram

CBOW

- Context words $\rightarrow$ center word.
- Averages context vectors.
- Fast; good for frequent words.

Skip-gram

- Center word $\rightarrow$ each context word.
- Many training pairs per position.
- Better for rare words; slower.

Example

"the cat drinks milk every day", window 1, target *drinks*. CBOW: (cat, milk) $\rightarrow$ drinks. Skip-gram: drinks $\rightarrow$ cat, drinks $\rightarrow$ milk.

Fast Training: Negative Sampling

- Replace the $|V|$-way softmax with binary classification: real pair vs. K random pairs.

$$\mathcal{L} = -\log \sigma(u_o^\top v_c) - \sum_{k=1}^{K} \log \sigma(-u_k^\top v_c)$$

- Negatives are drawn from the unigram distribution raised to the power 0.75.
- Subsampling of frequent words speeds training and improves rare-word vectors.
- Alternative: hierarchical softmax, a tree reducing cost to about $\log |V|$.

Key Takeaway

Word2Vec combined useful distributional representations with training objectives that scale to very large corpora.

Properties and Risks of Static Embeddings

Properties

- Neighbours show semantic similarity.
- Analogies sometimes work: king $-$ man $+$ woman $\approx$ queen (input words excluded).
- PCA/t-SNE visualize neighbourhoods.

Risks

- Inherit social bias from the corpus.
- Analogies are unreliable, not guaranteed.
- One vector per word type.

Important Note

Always ask which corpus, domain, period, and biases produced pretrained vectors.

- Evaluate embeddings intrinsically (similarity, neighbours, analogies) and, more importantly, extrinsically on the downstream task.

GloVe: Global Vectors (Pennington et al., 2014)

- Word2Vec learns from local prediction events generated by context windows.
- GloVe first accumulates explicit word–context co-occurrence statistics over the entire corpus, then learns vectors from those global statistics.
- Goal: dot products of vectors explain log co-occurrence.

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- f down-weights rare pairs and caps the influence of very frequent pairs.
- Ratios of co-occurrence probabilities encode informative semantic contrasts.

Important Note

GloVe is an explicit count-based embedding method; Word2Vec is prediction-based. Both learn static dense word vectors.

fastText: Subword Embeddings (Bojanowski et al., 2017)

- A word is the sum of its vector and its character n-gram vectors.
- *playing* includes <pl, pla, lay, ayi, ing, ng>; forms like *played* share pieces.
- **Pros:** vectors for OOV words and misspellings; good for morphologically rich languages such as Arabic.
- **Cons:** larger model; still a static representation for a word form in a given model.

Using Pretrained Embeddings

- Common sources: GloVe, fastText, Word2Vec vectors trained on large corpora.
- Align your vocabulary to the pretrained matrix; unknown words get a random or zero vector.
- Initialize the embedding layer with these weights.
- **Freeze** for small data; **fine-tune** when data is larger or the domain differs.

Key Takeaway

Pretrained embeddings transfer knowledge learned from large unlabeled corpora to downstream tasks. Implementation is covered in the lab.

From Word Vectors to Document Vectors

- **Average:** mean of word vectors; simple, loses order.
- **TF-IDF-weighted average:** emphasizes informative words.
- **SIF:** weighted average with common-component removal.
- **Doc2Vec:** learns a paragraph vector alongside word vectors.
- **Sentence encoders (e.g., Sentence-BERT):** sentence vectors for semantic search.

Important Note

Averaging treats "dog bites man" and "man bites dog" identically. Sequence models and transformers address this.

Static vs. Contextual Representations

- Static vectors give *bank* the same vector in "river bank" and "bank account".
- **ELMo (2018):** bidirectional LSTM states give a context-dependent vector per token.
- **BERT (2018):** self-attention over the whole sentence builds each token vector.
- Modern transformer tokenizers use subword units (e.g., BPE/WordPiece), greatly reducing the OOV problem.

Key Takeaway

Contextual models make token representations depend on surrounding text, addressing polysemy and order more directly, at a higher computational cost. Next lectures: RNNs, attention, transformers.

Comparison of Representations

Method	Type	Dim.	Semantics	Order/context	OOV	Main weakness
One-hot	Sparse	$\|V\|$	No	No	No	Huge, no similarity
BoW/TF-IDF	Sparse	$\|V\|$	No	Weak (n-grams)	No	Weak semantics
PPMI	Usually sparse	$\|V\|$ contexts	Yes	Window only	No	High-dimensional
LSA/SVD	Dense	k (e.g., 100–300)	Yes	No	No	Static; costly to update
Word2Vec	Dense	100–300	Yes	Window only	No	Static; OOV
GloVe	Dense	100–300	Yes	Window statistics	No	Static; OOV
fastText	Dense	100–300	Yes	Window only	Stronger	Static
ELMo/BERT	Dense	Model-dep.	Yes	Yes	Char./subword	More compute

Key Takeaway

Practical rule: start with TF-IDF + a linear model for a strong interpretable baseline; prefer fastText when morphology/OOV matters; use contextual encoders when ambiguity and word order are central.

Today's Key Concepts

1. Representation defines what a model can learn from text.
2. Sparse methods (one-hot, BoW, n-grams, TF-IDF) are strong baselines but ignore meaning.
3. The distributional hypothesis motivates count-based distributional representations such as co-occurrence/PPMI and dense reductions such as LSA/SVD.
4. Word2Vec learns vectors by prediction; negative sampling makes it scalable.
5. GloVe uses global counts; fastText adds subwords for OOV words.
6. Static vectors have one vector per word; contextual models fix this.

Key Takeaway

The progression is cumulative: identity $\rightarrow$ weighting $\rightarrow$ distributional similarity $\rightarrow$ scalable prediction $\rightarrow$ subwords/OOV handling $\rightarrow$ contextual meaning.

Lab 2 and Next Lecture

Lab 2 (hands-on code)

- Build BoW and TF-IDF features and train a baseline classifier.
- Train and explore Word2Vec; inspect neighbours and analogies.
- Use the PyTorch embedding layer with pretrained vectors (freeze vs. fine-tune).
- Compare TF-IDF, averaged embeddings, and a sentence encoder.

Lecture 3

RNNs, LSTM, and GRU for sequential data.

Discussion

- Why is TF-IDF a strong baseline despite ignoring meaning?
- When is fastText preferable to Word2Vec?
- For a small sentiment dataset, would you start with TF-IDF or pretrained embeddings?

Main Textbook Alignment

Primary Course Reference

Aurélien Géron (2025), *Hands-On Machine Learning with Scikit-Learn and PyTorch*, O'Reilly Media.

- **Chapter 14 – Natural Language Processing with RNNs and Attention:** embeddings, neural NLP inputs, sentiment analysis, pretrained tokenizers, pretrained embeddings and language models.
- **Chapter 15 – Transformers for Natural Language Processing and Chatbots:** transformer representations, BERT architecture, pretraining, and fine-tuning.
- **Chapter 7 – Dimensionality Reduction:** SVD/PCA and low-dimensional projection concepts that support the LSA discussion.

Questions

Questions?